

Introduction to Assembly Language

Delivered by: Belal Hashmi

BASIC COMPUTER ARCHITECTURE

- A computer system comprises of a processor, memory, and I/O devices.
- I/O is used for interfacing with the external world, while memory is the processor's internal world
- Processor is the core in this picture and is responsible for performing operations

Instruction Cycle

- The CPU fetches instructions from memory, reads and writes data from and to memory, and transfers data from and to input/output devices.
- A typical and simple execution cycle can be summarized as follows:
 - 1. The next instruction to be executed, whose address is obtained from the PC, is fetched from the memory and stored in the IR.
 - 2. The instruction is decoded.
 - 3. Operands are fetched from the memory and stored in CPU registers, if needed.
 - 4. The instruction is executed.
 - 5. Results are transferred from CPU registers to the memory, if needed.

Scenario

- Memory is a dumb device
- The scenario that the processor executes operations, and the memory contains data elements requires a mechanism for the processor to read that data from the memory

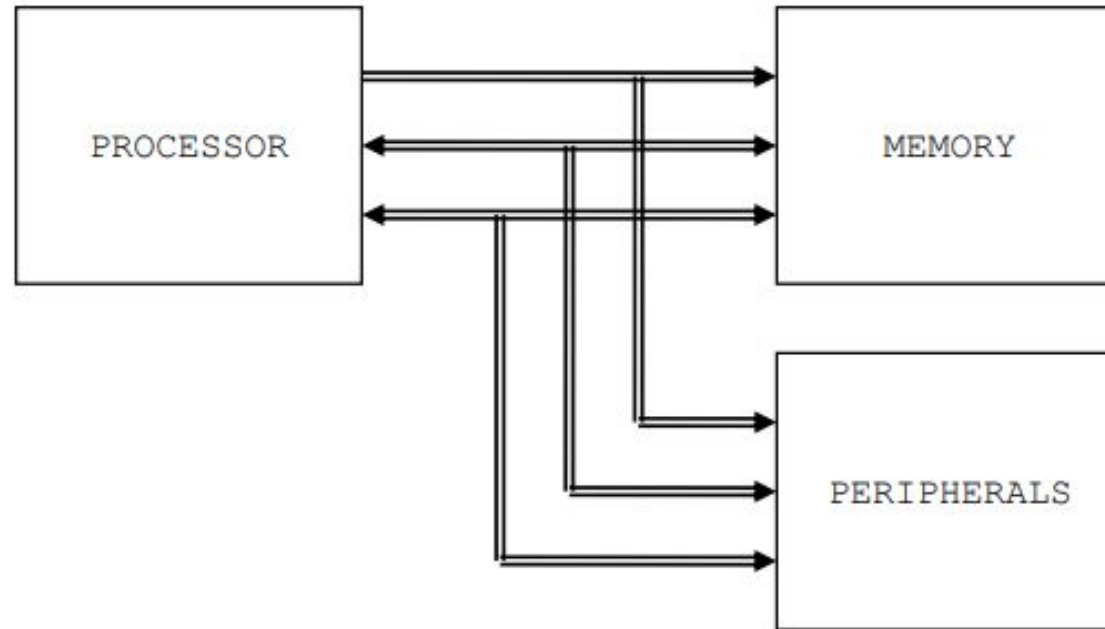
Key points

- There must be a mechanism to inform memory that we want to do the read operation
- There must be a mechanism to inform memory that we want to read precisely which element
- There must be a mechanism to transfer that data element from memory to processor

Buses

- The group of bits that the processor uses to inform the memory about which element to read or write is collectively known as the address bus.
- data bus is used to move the data from the memory to the processor in a read operation and from the processor to the memory in a write operation.
- Line of the bus is used to inform the memory about whether to do the read operation or the write operation. These lines are collectively known as the control bus

Buses



Buses

- The two dimensions, cell width and number of cells
- Word size of the memory(parallel bits)
- Same size of data bus and memory cell width.
- Control bus carries the intent of the processor
- Memory changes its behavior(direction of flow of data) in response to this signal from the processor.
- Precise synchronization between the processor and the memory is the responsibility of the control bus.

Buses

- why is the control bus bidirectional?
- Information cannot be transferred via the address or the data bus.
- Indicate the incapability of the peripheral to do something for the moment.

Registers

- operations need operands. One address bus and one data bus.
- Temporary storage places inside the processor called registers(precious resource)
- Sometimes we hold both operands in registers for the sake of efficiency
- Nomenclature(naming is useful since the registers are few in number)

Registers

- **Accumulator** :Central register in every processor called the accumulator. Traditionally all mathematical and logical operations are performed on the accumulator. The word size of a processor is defined by the width of its accumulator.
- **Pointer, Index, or Base Register**: Does not hold data but holds the address of data.
- **Flags Register or Program Status Word**: meaningless as a unit
- **Program Counter or Instruction Pointer**: The program counter holds the address of the next instruction to be executed.

Registers

- Opcode : one number tells it that it has to add, where its operands are, and where to store the result
- mnemonic : associate a symbol to every number

INSTRUCTION GROUPS

- Data Movement Instructions

mov ax, bx
lad 1234

- Arithmetic and Logic Instructions

and ax, 1234
add bx, 0534
add bx, [1200]

- Program Control Instructions

cmp ax, 0
jne 1234

- Special Instructions

cli
sti

INTEL IAPX88 ARCHITECTURE

- iAPX88 stands for “Intel Advanced Processor Extensions 88”
- 8088 is a 16bit processor
- downward compatible
- The architecture of a processor means the organization and functionalities of the registers
- History(64K on the 8085, the 8088 allowed a whole mega byte)

REGISTER ARCHITECTURE

The iAPX88 architecture consists of 14 registers.

CS	SP	
DS	BP	
SS	SI	
ES	DI	
	AH	AL (AX)
IP	BH	BL (BX)
	CH	CL (CX)
FLAGS	DH	DL (DX)

REGISTER ARCHITECTURE

- General Registers (AX, BX, CX, and DX) Accumulator, base, counter, destination
- Index Registers (SI and DI): allows mathematical and logical operations. implied functionality as the source or the destination in a special class of instructions called the string instructions. Not restricted to just string instructions. Cannot be used as 8 bit.
- Instruction Pointer (IP): address of the next instruction to be executed.
address of the next instruction to be executed
- Stack Pointer (SP): memory pointer
- Base Pointer (BP): memory pointer
- Flags Registers
- Segment Registers (CS, DS, SS, and ES)

Flag Registers

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				O	D	I	T	S	Z		A		P		C

Flags

C	Carry	When two 16bit numbers are added the answer can be 17 bits long or when two 8bit numbers are added the answer can be 9 bits long. This extra bit that won't fit in the target register is placed in the carry flag where it can be used and tested.
P	Parity	Parity is the number of "one" bits in a binary number. Parity is either odd or even. This information is normally used in communications to verify the integrity of data sent from the sender to the receiver.
A	Auxiliary Carry	A number in base 16 is called a hex number and can be represented by 4 bits. The collection of 4 bits is called a nibble. During addition or subtraction if a carry goes from one nibble to the next this flag is set. Carry flag is for the carry from the whole addition while auxiliary carry is the carry from the first nibble to the second.
Z	Zero Flag	The Zero flag is set if the last mathematical or logical instruction has produced a zero in its destination.

S	Sign Flag	A signed number is represented in its two's complement form in the computer. The most significant bit (MSB) of a negative number in this representation is 1 and for a positive number it is zero. The sign bit of the last mathematical or logical operation's destination is copied into the sign flag.
T	Trap Flag	The trap flag has a special role in debugging which will be discussed later.
I	Interrupt Flag	It tells whether the processor can be interrupted from outside or not. Sometimes the programmer doesn't want a particular task to be interrupted so the Interrupt flag can be zeroed for this time. The programmer rather than the processor sets this flag since the programmer knows when interruption is okay and when it is not. Interruption can be disabled or enabled by making this bit zero or one, respectively, using special instructions.
D	Direction Flag	Specifically related to string instructions, this flag tells whether the current operation has to be done from bottom to top of the block (D=0) or from top to bottom of the block (D=1).
O	Overflow Flag	The overflow flag is set during signed arithmetic, e.g. addition or subtraction, when the sign of the destination changes unexpectedly. The actual process sets the overflow flag whenever the carry into the MSB is different from the carry out of the MSB

Assembler, Linker, and Debugger

- “Netwide Assembler” or NASM.
- “A fullscreen debugger” or AFD

First Program

Example 1.1

```
001      ; a program to add three numbers using registers
002      [org 0x0100]
003          mov     ax, 5                ; load first number in ax
004          mov     bx, 10               ; load second number in bx
005          add     ax, bx               ; accumulate sum in ax
006          mov     bx, 15               ; load third number in bx
007          add     ax, bx               ; accumulate sum in ax
008
009          mov     ax, 0x4c00            ; terminate program
010          int     0x21
```

First Program Assembling

- To assemble we will give the following command to the processor assuming that our input file is named
- EX01.ASM. `nasm ex01.asm -o ex01.com -l ex01.lst`
- Produce two files EX01.COM that is our executable file and EX01.LST

List file

```
1
2
3 00000000 B80500
4 00000003 BB0A00
5 00000006 01D8
6 00000008 BB0F00
7 0000000B 01D8
8
9 0000000D B8004C
10 00000010 CD21
```

```
[org 0x0100]
mov ax, 5
mov bx, 10
add ax, bx
mov bx, 15
add ax, bx

mov ax, 0x4c00
int 0x21
```

First Program

- little-endian order and is used by Intel
- least significant, more significant, more significant, and most significant
- size of the complete COM file?
- 18 bytes

Intel way of writing

- The last three variations are for instructions that have one or both of their operands implied, or they work on a single or no operand. An implied operand means that it is always in a particular register say the accumulator, and it need not be mentioned in the instruction

```
operation destination, source  
operation destination  
operation source  
operation
```